

TAMIAS

Documento inicial de arquitectura y alcance MVP

Plataforma SaaS para la administración operativa de alojamientos pequeños

Stack principal	Angular, TypeScript, Java 21, Spring Boot 3, PostgreSQL, AWS S3, Spring AI
Arquitectura recomendada	Modular Monolith + Clean Architecture ligera
Modelo SaaS	Shared database + shared schema + organization_id
Despliegue objetivo	Vercel, Render, Supabase, Railway, AWS S3
Dominio	tamias.juantzun.dev

Versión inicial para guiar diseño, desarrollo, despliegue y evolución del producto.

1. Vision del producto

TAMIAS sera una plataforma SaaS para administrar la operacion diaria de alojamientos pequeños como casas vacacionales, apartamentos, bungalows, cabañas y villas. El objetivo es centralizar mantenimiento, reservaciones, tareas, compras, documentos, reportes y consultas asistidas con IA en una sola aplicacion.

La idea fuerte del producto no es solamente registrar datos, sino ayudar al administrador a responder preguntas operativas reales.

- ¿Cuando fue el ultimo mantenimiento del pozo?
- ¿Que tareas estan vencidas antes de una reservacion?
- ¿Que se compro la ultima vez para filtros de agua?
- ¿Que dice el reglamento sobre mascotas?
- ¿Donde esta ubicada la cisterna segun los planos?

Esto convierte a TAMIAS en una mezcla entre sistema administrativo y asistente operativo inteligente.

2. Alcance MVP recomendado

El alcance completo del producto es amplio. Para construirlo de forma realista y presentable en portfolio, conviene dividirlo en MVP y fases posteriores.

2.1 Modulos incluidos en el MVP

1. Authentication: login, JWT, roles basicos y usuario administrador inicial.
2. Organizations: cada organizacion representa a un propietario o administrador y es la base del modelo SaaS multi-tenant.
3. Properties: registro de propiedades, fotografias basicas y estado activo/inactivo.
4. Catalogs: categorias de mantenimiento, tipos, personas responsables, plataformas, proveedores, ciudades, materiales y suministros.
5. Maintenance: registro de mantenimientos realizados con categoria, tipo, fecha, costo, responsable, materiales usados e imagenes.
6. Scheduled Maintenance: mantenimientos programados con estados Scheduled, Completed, Rescheduled y Cancelled, razon de cancelacion/reprogramacion e historial basico.
7. Reservations: registro manual de reservaciones con plataforma, check-in, check-out, huespedes y observaciones.
8. Purchase Lists: listas de compra con proveedor, ciudad, items, cantidad, precio estimado y comprado/no comprado.
9. Documents: subida de documentos a S3, clasificacion por tipo y asociacion a propiedad.
10. AI Assistant basico: RAG sobre documentos con respuestas citando el documento fuente.
11. Deploy: frontend en Vercel, backend en Render, PostgreSQL en Supabase y dominio tamias.juantzun.dev.

2.2 Fuera del MVP inicial

- Recuperacion de contraseña.
- Invitaciones por correo.
- JasperReports y reportes avanzados.

- Tool calling completo contra PostgreSQL.
- Analisis de planos con Vision/OCR.
- Agentes IA especializados.
- Inventario formal.
- Notificaciones automaticas avanzadas.
- Multiples organizaciones con billing.
- Integracion directa con Airbnb, Booking o VRBO.

Estos puntos no se descartan; se posponen para evitar que el primer entregable sea demasiado grande.

3. Arquitectura general recomendada

La decision principal recomendada es usar un Modular Monolith con Clean Architecture ligera. No conviene iniciar con microservicios porque el producto tendra pocos usuarios simultaneos por organizacion, el dominio todavia evolucionara y desplegar muchos servicios aumentaria complejidad innecesaria.

Frontend Angular

|

| REST API + JWT

v

Backend Spring Boot 3

|

| JPA / Hibernate

v

PostgreSQL Supabase

Backend Spring Boot

|

| AWS SDK

v

AWS S3

Backend Spring Boot

|

| Spring AI

v

OpenAI / Ollama / Chroma

4. Estructura de repositorios

4.1 Opcion recomendada: Monorepo

```
tamias/  
  backend/  
  frontend/  
  docker-compose.yml  
  README.md  
  .github/  
    workflows/  
      backend-ci.yml  
      frontend-ci.yml
```

Ventajas: mas simple para portfolio, una sola URL de GitHub, documentacion centralizada y desarrollo local mas facil con Docker Compose. Desventajas: los pipelines deben filtrar cambios por carpeta y el repositorio puede crecer bastante.

4.2 Alternativa: repos separados

```
tamias-api/  
tamias-web/  
tamias-ai/
```

Ventajas: separacion mas profesional y despliegues independientes. Desventajas: mas mantenimiento, mas configuracion y mas friccion al inicio.

Recomendacion final: Monorepo, porque TAMIAS sera pieza principal de portfolio y permitira mostrar backend, frontend, Docker, CI/CD y documentacion en un solo lugar.

5. Modelo SaaS y multi-tenant

Aunque al inicio solo exista una organizacion, TAMIAS deberia nacer con estructura multi-tenant. La entidad raiz sera Organization.

```
Organization  
- id  
- name  
- status  
- created_at  
- updated_at
```

Casi todos los modulos deberan relacionarse con organization_id.

```
Organization  
|-- Users  
|-- Properties  
|-- Catalogs
```

```
|-- Reservations
|-- Maintenance Records
|-- Purchase Lists
|-- Documents
```

5.1 Estrategia recomendada

Usar shared database + shared schema + organization_id.

```
properties
- id
- organization_id
- name
- address
- status
```

Ventajas: simple, barato, ideal para Supabase y suficiente para pocas organizaciones. Desventaja: el backend debe filtrar siempre por organization_id para evitar fugas de datos.

```
WHERE organization_id = :currentOrganizationId
```

El backend debe obtener el organizationId desde el usuario autenticado, no desde el frontend.

6. Roles iniciales

Rol	Permisos principales	Restricciones sugeridas
Administrator	Gestion total de usuarios, propiedades, catalogos, mantenimiento, reservaciones, documentos, IA y reportes.	No aplica dentro de su organizacion.
Property Manager	Opera propiedades asignadas, mantenimientos, reservaciones, compras, documentos y reportes.	No gestiona roles criticos ni configuraciones globales.
Maintenance Staff	Ve mantenimientos asignados, completa tareas, sube evidencia y registra observaciones.	No ve finanzas completas ni administra usuarios/catalogos criticos.
Read Only	Consulta propiedades, reservaciones, mantenimiento, compras, documentos y reportes.	No modifica informacion.

7. Modulos backend recomendados

```
com.tamias
config
security
common
```

- organization
- user
- property
- catalog
- maintenance
- reservation
- purchase
- document
- notification
- report
- ai

Cada modulo puede organizarse internamente con controller, service, repository, entity, dto, mapper y exception.

- maintenance
 - controller
 - MaintenanceRecordController.java
 - ScheduledMaintenanceController.java
 - service
 - MaintenanceRecordService.java
 - ScheduledMaintenanceService.java
 - repository
 - MaintenanceRecordRepository.java
 - ScheduledMaintenanceRepository.java
 - entity
 - MaintenanceRecord.java
 - ScheduledMaintenance.java
 - MaintenanceHistory.java
 - dto
 - MaintenanceRecordRequest.java
 - MaintenanceRecordResponse.java

8. Entidades principales iniciales

8.1 Seguridad y organizacion

- Organization
- User
- Role
- Permission
- UserOrganization

Para MVP podria simplificarse a User, Role y Organization, pero UserOrganization permite que un usuario pertenezca a mas de una organizacion en el futuro.

8.2 Propiedades

Property

PropertyImage

Property

- id
- organization_id
- name
- address
- description
- status
- created_at
- updated_at

8.3 Catalogos

Recomendacion: usar tablas separadas para catalogos importantes porque reflejan mejor el dominio y facilitan reglas futuras.

MaintenanceCategory

MaintenanceType

Supplier

City

Platform

Material

TaskTemplate

8.4 Mantenimiento

MaintenanceRecord

MaintenanceRecordImage

MaintenanceMaterialUsed

ScheduledMaintenance

ScheduledMaintenanceHistory

MaintenanceResponsiblePerson

8.5 Reservaciones

Reservation

Guest

ReservationGuest

Platform

Para MVP se pueden guardar huespedes como texto, pero Guest es mas escalable.

8.6 Compras

PurchaseList
PurchaseItem
Supplier
City
Material
Brand

8.7 Documentos

Document
DocumentChunk
DocumentEmbedding

Si se usa Chroma, los embeddings vivirían en Chroma y PostgreSQL guardaría metadatos.

Document

- id
- organization_id
- property_id
- document_type
- original_filename
- s3_key
- content_type
- size
- status
- uploaded_at

9. Base de datos inicial

Reglas recomendadas: usar UUID, organization_id en tablas multi-tenant, created_at y updated_at, evitar borrado físico para información crítica y usar status o deleted_at donde aplique.

organizations
users
roles
user_organizations
properties
property_images
maintenance_categories
maintenance_types
maintenance_people
maintenance_records
maintenance_record_images

maintenance_materials_used
scheduled_maintenance
scheduled_maintenance_history
reservations
guests
reservation_guests
platforms
purchase_lists
purchase_items
suppliers
cities
materials
brands
documents

10. Estrategia de APIs REST

Usar rutas versionadas para mantener estabilidad a futuro.

/api/v1/auth
/api/v1/users
/api/v1/organizations
/api/v1/properties
/api/v1/catalogs
/api/v1/maintenance-records
/api/v1/scheduled-maintenance
/api/v1/reservations
/api/v1/purchase-lists
/api/v1/documents
/api/v1/ai

10.1 Ejemplos

GET /api/v1/properties
GET /api/v1/properties/{id}
POST /api/v1/properties
PUT /api/v1/properties/{id}
DELETE /api/v1/properties/{id}

GET /api/v1/maintenance-records
GET /api/v1/maintenance-records/{id}
POST /api/v1/maintenance-records

```
PUT    /api/v1/maintenance-records/{id}
DELETE /api/v1/maintenance-records/{id}
POST   /api/v1/maintenance-records/{id}/images
```

```
POST /api/v1/ai/chat
POST /api/v1/ai/documents/search
POST /api/v1/ai/tools/query
```

11. Frontend Angular

```
src/app
  core
    auth
    interceptors
    guards
    layout
  shared
    components
    pipes
    validators
  features
    dashboard
    properties
    maintenance
    reservations
    purchases
    documents
    ai-assistant
    catalogs
    users
```

Principios: Angular standalone components, Reactive Forms, servicios por feature, interceptors para JWT, guards por autenticacion y rol, componentes reutilizables y Bootstrap para acelerar UI.

11.1 Pantallas MVP

- Login
- Dashboard
- Propiedades
- Formulario de propiedad
- Mantenimientos
- Formulario de mantenimiento
- Mantenimientos programados

- Calendario de mantenimientos
- Reservaciones
- Compras
- Documentos
- Asistente IA
- Catalogos

12. IA en TAMIAS

12.1 IA Fase 1 - RAG sobre documentos

Objetivo: permitir preguntas sobre reglamentos, manuales, señales y documentos importantes, devolviendo respuestas con fuente.

```
Usuario sube documento
|
Backend guarda archivo en S3
|
Backend extrae texto
|
Texto se divide en chunks
|
Chunks se vectorizan
|
Embeddings se guardan en Chroma
|
Usuario pregunta
|
Se busca contexto relevante en Chroma
|
OpenAI genera respuesta
|
Respuesta cita documento fuente
```

12.2 IA Fase 2 - Tool Calling

Objetivo: que la IA consulte datos reales del sistema sin ejecutar SQL libre directamente. Lo correcto es exponer herramientas controladas.

```
findLastPurchaseByMaterial(materialName)
getMaintenanceCostByYear(year)
findOverdueTasks()
findLastMaintenanceByCategory(categoryName)
```

12.3 IA Fase 3 - Blueprint Analysis

Objetivo: analizar planos, medidas y ubicaciones con OCR y modelos de vision. Cuando la informacion no sea concluyente, el sistema debe indicarlo claramente.

No puedo determinarlo con certeza a partir del plano proporcionado.

Segun el plano, la habitacion principal parece medir aproximadamente 4.2 m x 3.8 m, pero la imagen no permite confirmarlo con total precision.

13. Reporteria

Para MVP no se recomienda incluir JasperReports todavia, pero si conviene dejar diseñada la arquitectura.

```
report
  controller
  service
  templates
```

- Maintenance History
- Maintenance Costs
- Upcoming Maintenance
- Reservation Summary
- Purchase History
- Expense Summary
- Inventory Usage
- Task Completion

```
GET /api/v1/reports/maintenance-history?propertyId=...&from=...&to=...
```

Response: application/pdf

14. Notificaciones

En MVP se pueden dejar notificaciones basicas por correo para invitaciones, mantenimiento proximo, tareas vencidas y reservaciones creadas.

```
NotificationService
EmailService
EmailTemplateService
```

Mas adelante se pueden agregar colas, recordatorios programados, plantillas HTML e historial de notificaciones.

15. Seguridad

15.1 JWT

- Access token corto.

- Refresh token opcional para fase posterior.
- Roles en claims.
- Permisos validados en backend, no solo en frontend.

15.2 Multi-tenant

Nunca confiar en organizationId enviado por el frontend para filtrar datos criticos.

JWT -> userId -> organizationId actual -> filtrar datos

15.3 Archivos

- No exponer rutas internas de S3.
- Usar pre-signed URLs.
- Validar tipo y tamaño de archivo.
- Separar archivos por organizacion y propiedad.

organizations/{organizationId}/properties/{propertyId}/documents/{documentId}/file.pdf

15.4 IA

La IA debe respetar permisos. El tool calling debe filtrar por organizacion y rol, y no debe permitir acciones no autorizadas.

16. DevOps

16.1 Docker Compose local

```
frontend
backend
postgres
chroma
ollama
```

16.2 GitHub Actions

Push/PR

```
|
|-- Backend tests
|-- Frontend tests
|-- Backend build
|-- Frontend build
|-- Docker build
```

Merge a main

```
|
|-- Build
|-- Tests
|-- Docker Build
```

| -- Deploy Backend Render
| -- Deploy Frontend Vercel

17. Despliegue recomendado

Componente	Servicio recomendado
Frontend	Vercel
Backend	Render
Base de datos	Supabase PostgreSQL
Vector DB / IA	Railway
Archivos	AWS S3
Dominio	tamias.juantzun.dev

Para MVP se puede iniciar con frontend en Vercel, backend en Render, base de datos en Supabase, archivos en AWS S3 y OpenAI API con Chroma en Railway. Ollama conviene dejarlo principalmente para ambiente local.

18. Roadmap por fases

Fase	Objetivo
Fase 0	Preparacion: repositorio, README, monorepo, backend, frontend, Docker Compose, PostgreSQL local y Flyway.
Fase 1	Seguridad y base SaaS: Organization, User, roles, login, JWT, seed admin, guards e interceptor.
Fase 2	Propiedades y catalogos: CRUDs de propiedades, categorias, tipos, proveedores, ciudades, plataformas y materiales.
Fase 3	Mantenimiento: registros, materiales, responsables, imagenes, programacion, reprogramacion, cancelacion e historial.
Fase 4	Reservaciones y tareas: CRUD de reservaciones, tareas por reservacion, catalogo de tareas y checklist.
Fase 5	Compras: listas de compra, items, proveedor, ciudad, material, precio estimado y comprado/no comprado.
Fase 6	Documentos: subida a S3, tipos, asociacion a propiedad, descarga segura y pre-signed URLs.
Fase 7	IA RAG: extraccion de texto, chunks, embeddings, consulta y respuestas con fuente.
Fase 8	Deploy: Render, Vercel, Supabase, S3, variables de entorno y dominio.
Fase 9	Tool Calling: herramientas controladas para compras,

	mantenimiento, tareas y reservaciones.
Fase 10	Reportes y notificaciones: JasperReports, PDFs, correos y recordatorios.
Fase 11	Blueprint Analysis: OCR, vision models, interpretacion de planos e incertidumbre controlada.

19. Decisiones tecnicas iniciales

Area	Decision
Backend	Java 21, Spring Boot 3, Spring Security, JWT, Spring Data JPA, Hibernate, Flyway, PostgreSQL y Swagger/OpenAPI.
Frontend	Angular, TypeScript, Bootstrap, Reactive Forms y FullCalendar.
IA	Spring AI, OpenAI, Chroma, RAG y Tool Calling. Ollama como complemento local.
Archivos	AWS S3 para documentos, imagenes y evidencias de mantenimiento.

20. Primera decision importante

20.1 TAMIAS MVP incluye

Authentication
Organizations
Users
Properties
Catalogs
Maintenance
Scheduled Maintenance
Reservations
Purchase Lists
Documents
AI Document Search
Basic Deploy

20.2 TAMIAS v2 incluye

Advanced Notifications
JasperReports
Tool Calling

- Task Automation
- Advanced Dashboard
- Blueprint Analysis

20.3 TAMIAS v3 incluye

- Agents
- Inventory
- Mobile-friendly workflows
- External platform integrations
- Billing/subscriptions
- Advanced analytics

21. Siguiente entregable recomendado

El siguiente paso recomendado es diseñar la base de datos MVP, bajando de arquitectura a algo implementable: tablas, campos, relaciones, indices, constraints, soft delete, migraciones Flyway y separacion entre MVP y fases futuras.

- organizations
- users
- roles
- permissions
- properties
- maintenance_categories
- maintenance_types
- maintenance_records
- scheduled_maintenance
- reservations
- purchase_lists
- documents